

Table of Contents

Comprehensive Question Bank: Unit 3 — Algorithms and Problem Solving	4
Section A: Multiple-Choice Questions (MCQs).....	4
Topic 3.1: Understanding Computational Problems	4
Topic 3.1.1: Characterizing Computational Problems.....	4
Topic 3.1.1.1: Classifying Computational Problems	5
Topic 3.1.1.2: Well-defined vs. Ill-defined Problems	6
Topic 3.2: Algorithms for Problem Solving	7
Topic 3.2.1: Generate-and-Test Method	7
Topic 3.3: Problem Solvability and Complexity	8
Topic 3.3.1: Solvable vs. Unsolvable Problems.....	9
Topic 3.3.2: Tractable vs. Intractable Problems.....	9
Topic 3.3.3: Complexity Classes (P, NP, NP-hard, NP-complete).....	10
Topic 3.3.3.1: Class P	11
Topic 3.3.3.2: Class NP	11
Topic 3.3.3.3: Class NP-Hard	12
Topic 3.3.3.4: NP-Complete.....	12
Topic 3.4: Algorithm Analysis	13
Topic 3.4.1: Time Complexity.....	13
Topic 3.4.1.1: Big O Notation	13
Topic 3.4.2: Space Complexity	15
Topic 3.5: Algorithm Design Techniques.....	16
Topic 3.5.1: Divide and Conquer	16
Topic 3.5.2: Greedy Algorithms.....	16
Topic 3.5.3: Dynamic Programming.....	17
Topic 3.5.4: Backtracking.....	18
Topic 3.6: Commonly Used Algorithms.....	18
Topic 3.6.1: Sorting Algorithms	19
Topic 3.6.1.1: Bubble Sort	19
Topic 3.6.1.2: Selection Sort.....	20
Topic 3.6.2: Search Algorithms	21
Topic 3.6.2.1: Linear Search.....	21
Topic 3.6.2.2: Binary Search.....	21
Topic 3.6.3: Graph Algorithms	23
Topic 3.6.3.1: Breadth-First Search (BFS).....	23

Topic 3.6.3.2: Depth-First Search (DFS)	24
Section B: Short Answer Questions (SQs)	25
Topic 3.1: Understanding Computational Problems	25
Topic 3.1.1: Characterizing Computational Problems	25
Topic 3.1.1.1: Classifying Computational Problems	25
Topic 3.1.1.2: Well-defined vs. Ill-defined Problems	25
Topic 3.2: Algorithms for Problem Solving	25
Topic 3.2.1: Generate-and-Test Method	26
Topic 3.3: Problem Solvability and Complexity	26
Topic 3.3.1: Solvable vs. Unsolvable Problems	26
Topic 3.3.2: Tractable vs. Intractable Problems	26
Topic 3.3.3: Complexity Classes (P, NP, NP-hard, NP-complete)	26
Topic 3.3.3.1: Class P	26
Topic 3.3.3.2: Class NP	27
Topic 3.3.3.3: Class NP-Hard	27
Topic 3.3.3.4: NP-Complete	27
Topic 3.4: Algorithm Analysis	27
Topic 3.4.1: Time Complexity	27
Topic 3.4.1.1: Big O Notation	27
Topic 3.4.2: Space Complexity	27
Topic 3.5: Algorithm Design Techniques	28
Topic 3.5.1: Divide and Conquer	28
Topic 3.5.2: Greedy Algorithms	28
Topic 3.5.3: Dynamic Programming	28
Topic 3.5.4: Backtracking	28
Topic 3.6: Commonly Used Algorithms	28
Topic 3.6.1: Sorting Algorithms	28
Topic 3.6.1.1: Bubble Sort	29
Topic 3.6.1.2: Selection Sort	29
Topic 3.6.2: Search Algorithms	29
Topic 3.6.2.1: Linear Search	29
Topic 3.6.2.2: Binary Search	29
Topic 3.6.3: Graph Algorithms	29
Topic 3.6.3.1: Breadth-First Search (BFS)	30
Topic 3.6.3.2: Depth-First Search (DFS)	30
Section C: Long / Extensive Questions (LQs)	30
Topic 3.1: Understanding Computational Problems	30

Topic 3.1.1 & 3.1.1.1: Characterizing and Classifying Computational Problems	30
Topic 3.1.1.2: Well-defined vs. Ill-defined Problems	30
Topic 3.2 & 3.2.1: Algorithms and the Generate-and-Test Method	31
Topic 3.3: Problem Solvability and Complexity	31
Topic 3.3.3: Complexity Classes (P, NP, NP-hard, NP-complete).....	31
Topic 3.4: Algorithm Analysis (Time and Space Complexity)	32
Topic 3.5: Algorithm Design Techniques	32
Topic 3.6.1: Sorting Algorithms	33
Topic 3.6.2: Search Algorithms	33
Topic 3.6.3: Graph Algorithms	33

Comprehensive Question Bank: Unit 3 — Algorithms and Problem Solving

Section A: Multiple-Choice Questions (MCQs)

Topic 3.1: Understanding Computational Problems

1. A computational problem is best defined as:
A) Any error that occurs while running a program
B) A challenge that can be solved through a computational process using an algorithm
C) A hardware malfunction in a computer system
D) A syntax error in source code
 2. Which of the following correctly identifies the three essential parts of a computational problem?
A) Actor, Class, Method
B) Input, Process, Output
C) Requirement, Design, Testing
D) Compiler, Interpreter, Debugger
 3. In a computational problem, the "Process" refers to:
A) The data given to the algorithm at the start
B) The steps or rules applied to the input to generate the output
C) The final result produced by the algorithm
D) The hardware used to run the algorithm
 4. In a computational problem, the "Output" refers to:
A) The data given to the algorithm at the beginning
B) The steps applied to transform data
C) The solution or result produced after processing the input
D) The programming language used to write the algorithm
-

Topic 3.1.1: Characterizing Computational Problems

1. To solve a problem computationally, which of the following must first be identified?
A) Only the programming language to be used
B) The inputs, desired outputs, and the process needed to transform inputs into

outputs

- C) Only the hardware requirements
 - D) Only the final deadline for the project
2. Characterizing a computational problem is most similar to which activity?
- A) Randomly writing code without a plan
 - B) Clearly identifying what a system needs before attempting to build a solution**
 - C) Installing software on a user's device
 - D) Marketing a finished software product
-

Topic 3.1.1.1: Classifying Computational Problems

1. A problem where the output is a simple "yes" or "no" is classified as a:
- A) Decision Problem**
 - B) Search Problem
 - C) Optimization Problem
 - D) Counting Problem
2. A problem where the task is to find a solution or item meeting certain criteria is classified as a:
- A) Decision Problem
 - B) Search Problem**
 - C) Optimization Problem
 - D) Counting Problem
3. A problem where the goal is to find the best solution according to some criteria is classified as a:
- A) Decision Problem
 - B) Search Problem
 - C) Optimization Problem**
 - D) Counting Problem
4. A problem where the objective is to count the number of ways certain conditions can be met is classified as a:
- A) Decision Problem
 - B) Search Problem
 - C) Optimization Problem
 - D) Counting Problem**
5. "Is this number prime?" is an example of which type of computational problem?
- A) Decision Problem**
 - B) Optimization Problem
 - C) Counting Problem
 - D) Search Problem

6. "Find the cheapest flight from Lahore to Dubai" is an example of which type of computational problem?
- A) Decision Problem
 - B) Search Problem
 - C) Optimization Problem**
 - D) Counting Problem
7. "How many different ways can you make change for 100 rupees?" is an example of which type of computational problem?
- A) Decision Problem
 - B) Search Problem
 - C) Optimization Problem
 - D) Counting Problem**
-

Topic 3.1.1.2: Well-defined vs. Ill-defined Problems

1. A well-defined problem is characterized by:
- A) Ambiguous goals and unclear requirements
 - B) Clear goals, inputs, processes, and outputs**
 - C) A vague, subjective solution
 - D) No measurable stopping point
2. "Determine if a number is even" is considered a well-defined problem because:
- A) It has no clear input
 - B) It has a clear goal, input, process, and output**
 - C) It cannot be solved by a computer
 - D) It requires human judgment to complete
3. "How to reduce poverty in Pakistan" is an example of a(n):
- A) Well-defined problem
 - B) Ill-defined problem**
 - C) Decision problem
 - D) Tractable problem
4. Which of the following best describes an ill-defined problem?
- A) A problem with a clear, measurable output
 - B) A problem that lacks clear definitions or has ambiguous goals and requirements**
 - C) A problem that can always be solved in polynomial time
 - D) A problem verified quickly once a solution is given

5. In the flowchart "Finding an Even Number" (Figure 3.1), what is the clear input to the problem?
- A) The word "Even" or "Odd"
 - B) A single integer**
 - C) A list of numbers
 - D) A sorted array
-

Topic 3.2: Algorithms for Problem Solving

1. An algorithm is best described as:
 - A) A random guess at a solution
 - B) A step-by-step procedure for solving a problem**
 - C) A type of computer hardware
 - D) A finished software product
 2. Why are algorithms considered essential in software development?
 - A) They eliminate the need for testing
 - B) They provide the logic behind software operations, helping solve problems, optimize performance, and ensure accuracy**
 - C) They replace the need for programming languages
 - D) They are only used in artificial intelligence applications
 3. Which algorithm does Google use to determine the relevance of web pages in search results?
 - A) Binary Search
 - B) PageRank**
 - C) QuickSort
 - D) Dijkstra's Algorithm
 4. According to this unit, PageRank ranks web pages based on:
 - A) The color scheme of the website
 - B) The number of links to a page and the quality of those links**
 - C) The length of the webpage's text
 - D) The webpage's loading speed only
-

Topic 3.2.1: Generate-and-Test Method

1. The Generate-and-Test method works by:
 - A) Designing the most efficient solution mathematically before testing
 - B) Generating potential solutions and testing each one until a satisfactory solution is found**
 - C) Skipping testing entirely to save time
 - D) Only generating a single possible solution

2. The Generate-and-Test method is particularly useful when:
 - A) The problem space is extremely large and clearly structured
 - B) The problem space is small, or there is no clear strategy for finding a solution**
 - C) A solution is already known in advance
 - D) The problem requires no testing at all
 3. In the Generate-and-Test flowchart, what happens after the Tester identifies an "Incorrect Solution"?
 - A) The process stops immediately
 - B) The process returns to the Generator to produce another possible solution**
 - C) The incorrect solution is used as the final answer
 - D) The problem is reclassified as unsolvable
 4. Trying every combination on a forgotten 4-digit bike lock one by one is an example of which method?
 - A) Divide and Conquer
 - B) Generate-and-Test**
 - C) Dynamic Programming
 - D) Backtracking
 5. According to this unit, the Generate-and-Test method is often used in which field, particularly for game playing and problem-solving?
 - A) Database management
 - B) Artificial Intelligence (AI)**
 - C) Network security
 - D) Graphic design
-

Topic 3.3: Problem Solvability and Complexity

1. Problem solvability and complexity primarily help determine:
 - A) The programming language best suited for a problem
 - B) Whether a problem can be solved using an algorithm, and if so, how efficiently**
 - C) The cost of developing a software solution
 - D) The visual design of a software system
-

Topic 3.3.1: Solvable vs. Unsolvable Problems

1. A problem is considered solvable if:
 - A) It can be verified quickly once a solution is proposed
 - B) An algorithm can solve it within a finite amount of time**
 - C) It requires exponential time to solve
 - D) It has no clearly defined inputs or outputs
 2. Calculating the Greatest Common Divisor (GCD) of two integers using the Euclidean algorithm is an example of a:
 - A) Solvable problem**
 - B) Unsolvable problem
 - C) Ill-defined problem
 - D) NP-hard problem
 3. The Halting Problem asks:
 - A) Whether a given number is prime
 - B) Whether a given program will eventually halt or run forever**
 - C) Whether a list of numbers is sorted
 - D) Whether two graphs are identical
 4. Who proved that the Halting Problem is unsolvable, and in what year?
 - A) Stephen Cook, in 1971
 - B) Alan Turing, in 1936**
 - C) Richard Bellman, in the 1950s
 - D) Leonhard Euler, in 1736
 5. A problem is considered unsolvable if:
 - A) It can always be solved in polynomial time
 - B) No algorithm can be created that provides a solution in all cases**
 - C) It belongs to Class P
 - D) Its solution can be verified quickly
-

Topic 3.3.2: Tractable vs. Intractable Problems

1. A tractable problem is one that can be solved in:
 - A) Exponential time
 - B) Polynomial time**
 - C) No time at all
 - D) Undefined time

2. Sorting a list of numbers using Merge Sort or Quick Sort, with a time complexity of $O(n \log n)$, is an example of a:
- A) Tractable problem**
 - B) Intractable problem
 - C) Unsolvable problem
 - D) Ill-defined problem
3. An intractable problem is best described as one that:
- A) Can always be solved instantly regardless of input size
 - B) Requires super-polynomial time, often growing exponentially with input size**
 - C) Has no defined input or output
 - D) Can never be verified once solved
4. The Traveling Salesman Problem (TSP) is considered intractable because:
- A) It has no possible solution
 - B) The number of possible routes grows factorially as the number of cities increases**
 - C) It can be solved in constant time
 - D) It is a well-defined but simple problem
5. Which classification best applies to the Traveling Salesman Problem?
- A) Class P
 - B) NP-hard**
 - C) Well-defined but trivial
 - D) $O(1)$ constant time
-

Topic 3.3.3: Complexity Classes (P, NP, NP-hard, NP-complete)

1. In 1971 and 1973, which two computer scientists independently formulated the P vs. NP question?
- A) Alan Turing and Richard Bellman
 - B) Stephen Cook and Leonid Levin**
 - C) Grady Booch and Ivar Jacobson
 - D) Linus Torvalds and Winston Royce
2. What prize is offered by the Clay Mathematics Institute for solving the P vs. NP problem?
- A) \$500,000
 - B) \$1,000,000 Millennium Prize**
 - C) \$10,000
 - D) No prize is currently offered
3. According to the Venn diagram of complexity classes (Figure 3.4), which class is shown as a subset contained within NP?
- A) NP-hard**

B) P and NP-Complete

C) The Halting Problem

D) Ill-defined problems

4. Why is the question of whether P equals NP considered so significant?

A) It only affects theoretical mathematics with no real-world impact

B) It has major implications for cryptography, algorithm design, and computational complexity

C) It has already been definitively answered

D) It only applies to sorting algorithms

Topic 3.3.3.1: Class P

1. Class P refers to problems that:

A) Can only be verified quickly, not solved quickly

B) Can be solved efficiently by a computer, even as the problem size grows

C) Have no known algorithm at all

D) Always require exponential time

2. Sorting a list such as [4, 1, 3, 2, 5] into ascending order is an example of a problem in:

A) Class P

B) Class NP-hard only

C) The Unsolvable class

D) The Ill-defined class

Topic 3.3.3.2: Class NP

1. Class NP refers to problems where:

A) No solution can ever be verified

B) A proposed solution can be checked quickly, even if finding it is difficult

C) A solution can never be found or verified

D) The problem is always solvable in constant time

2. Solving a Sudoku puzzle is often used as an example of Class NP because:

A) Sudoku puzzles cannot be solved at all

B) While solving it may be hard, checking a completed solution is quick and easy

C) It always takes constant time to solve

D) It has no defined rules

Topic 3.3.3.3: Class NP-Hard

1. NP-hard problems are defined as problems that are:
 - A) Easier than any problem in Class P
 - B) At least as difficult as the hardest problems in NP**
 - C) Always solvable in polynomial time
 - D) Impossible to define computationally
 2. Which of the following is a well-known example of an NP-hard problem, as discussed in this unit?
 - A) Sorting a list of five numbers
 - B) The Traveling Salesman Problem (TSP)**
 - C) Determining if a number is even
 - D) Linear Search
-

Topic 3.3.3.4: NP-Complete

1. NP-Complete problems are defined as problems that are:
 - A) Only in Class P, never in NP
 - B) Both in NP and as hard as the hardest problems in NP**
 - C) Always unsolvable
 - D) Never able to be verified quickly
 2. If an efficient algorithm were found for just one NP-Complete problem, what would this imply?
 - A) Nothing — it would only apply to that one specific problem
 - B) All NP problems could then be solved efficiently**
 - C) The problem would be reclassified as unsolvable
 - D) The problem would move to Class P-hard
 3. Which of the following is a classic example of an NP-Complete problem discussed in this unit?
 - A) Bubble Sort
 - B) The Knapsack Problem**
 - C) Linear Search
 - D) Breadth-First Search
 4. In the Knapsack Problem, the objective is to:
 - A) Sort items by weight only
 - B) Determine the most valuable combination of items that fits within a weight capacity**
 - C) Find the shortest path between two cities
 - D) Count the number of items in a list
-

Topic 3.4: Algorithm Analysis

1. Algorithm analysis is best defined as:
 - A) The process of writing new algorithms from scratch
 - B) The process of determining the computational complexity of an algorithm, including time and space complexity**
 - C) The process of testing software for bugs
 - D) The process of gathering user requirements
 2. Why is algorithm analysis considered crucial in software development?
 - A) It replaces the need for writing code
 - B) It helps predict an algorithm's performance and supports selecting the best algorithm for a task**
 - C) It eliminates the need for sorting algorithms
 - D) It is only useful for graph traversal problems
-

Topic 3.4.1: Time Complexity

1. Time complexity is best defined as a measure of:
 - A) The amount of memory an algorithm consumes
 - B) How the running time of an algorithm increases as the input size grows**
 - C) The number of programmers required for a project
 - D) The visual complexity of a program's user interface
 2. As the volume of numbers in a list increases, what generally happens to the time required to sort them?
 - A) The time required always decreases
 - B) The time required also increases**
 - C) The time required stays exactly the same regardless of algorithm
 - D) The time required becomes undefined
-

Topic 3.4.1.1: Big O Notation

1. Big O notation is best described as:
 - A) A programming language used for algorithm design
 - B) A mathematical way to describe the time complexity of an algorithm, providing an upper bound on runtime**
 - C) A type of sorting algorithm
 - D) A graph traversal technique

2. Which Big O notation represents "Constant Time," where runtime stays the same regardless of input size?
- A) $O(1)$**
 - B) $O(n)$
 - C) $O(n^2)$
 - D) $O(\log n)$
3. Which Big O notation represents "Linear Time," where runtime grows directly with input size?
- A) $O(1)$
 - B) $O(n)$**
 - C) $O(n^2)$
 - D) $O(\log n)$
4. Which Big O notation represents "Quadratic Time," where runtime grows with the square of the input size?
- A) $O(1)$
 - B) $O(n)$
 - C) $O(n^2)$**
 - D) $O(\log n)$
5. Which Big O notation represents "Logarithmic Time," where runtime grows very slowly relative to input size?
- A) $O(1)$
 - B) $O(n)$
 - C) $O(n^2)$
 - D) $O(\log n)$**
6. Searching through a list of n students one by one to find a specific student ID has a time complexity of:
- A) $O(1)$
 - B) $O(n)$**
 - C) $O(n^2)$
 - D) $O(\log n)$
7. Comparing every pair of n students to determine the best team has a time complexity of approximately:
- A) $O(1)$
 - B) $O(n)$
 - C) $O(n^2)$**
 - D) $O(\log n)$
8. Guessing a number between 1 and 100 using yes/no questions that repeatedly cut the range in half demonstrates which time complexity?
- A) $O(1)$**

- B) $O(n)$
- C) $O(n^2)$
- D) $O(\log n)$**

9. According to Figure 3.5, which time complexity grows the fastest as input size increases?

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n^2)$**

10. Big O notation is primarily used to describe an algorithm's performance in which scenario?

- A) The best-case scenario only
- B) The worst-case scenario**
- C) A scenario with zero input
- D) A scenario unrelated to input size

11. In algorithm analysis, what is the first practical step recommended in this unit for estimating an algorithm's Big O complexity?

- A) Measure the algorithm's exact runtime in seconds
- B) Count the number of loops and whether they are nested**
- C) Estimate the algorithm's memory usage
- D) Determine whether the algorithm is written in Python or C++

Topic 3.4.2: Space Complexity

1. Space complexity measures:

- A) How an algorithm's runtime changes as input size grows
- B) How the amount of memory an algorithm uses changes as input size grows**
- C) The number of lines of code in an algorithm
- D) The number of developers needed to implement an algorithm

2. If an algorithm needs to store a growing list of numbers, its space complexity indicates:

- A) How quickly the algorithm will run
 - B) How much memory will be required as the volume of numbers increases**
 - C) How many bugs the algorithm is likely to contain
 - D) How many times the algorithm must be tested
-

Topic 3.5: Algorithm Design Techniques

1. Algorithm design techniques are best described as:
 - A) One-time tricks that apply to only a single specific problem
 - B) Reusable, systematic strategies for solving problems efficiently and effectively**
 - C) A category of UML diagrams
 - D) A method for gathering software requirements
-

Topic 3.5.1: Divide and Conquer

1. Divide and Conquer works by:
 - A) Solving the entire problem at once without breaking it down
 - B) Breaking a large problem into smaller parts, solving each independently, and combining the solutions**
 - C) Making a sequence of locally optimal choices
 - D) Storing results of subproblems to avoid recalculation
 2. Which sorting algorithm is used as the primary example of Divide and Conquer in this unit?
 - A) Bubble Sort
 - B) Selection Sort
 - C) Merge Sort**
 - D) Linear Search
 3. Divide and Conquer is particularly effective for problems that:
 - A) Cannot be broken into smaller parts
 - B) Can be divided into similar smaller sub-problems**
 - C) Require no combination of results
 - D) Have only one possible solution
-

Topic 3.5.2: Greedy Algorithms

1. Greedy algorithms work by:
 - A) Testing every possible solution before selecting one
 - B) Making a sequence of locally optimal choices, hoping they lead to a globally optimal solution**
 - C) Breaking a problem into smaller, independent sub-problems
 - D) Storing all previous results in a memoization table
2. The Coin Change problem, where the largest denomination coin not exceeding the remaining amount is chosen repeatedly, is a classic example of a:
 - A) Divide and Conquer algorithm

B) Greedy algorithm

C) Dynamic Programming algorithm

D) Backtracking algorithm

3. The greedy approach is often most appropriate when a problem has:

A) No relationship between its sub-problems

B) Optimal substructure, where the best overall solution can be built from optimal sub-solutions

C) Only unsolvable sub-problems

D) A single possible input value

4. According to this unit, what is an important limitation of greedy algorithms?

A) They are always slower than other techniques

B) They don't always guarantee the optimal solution for every problem

C) They cannot be implemented in any programming language

D) They require exponential memory in every case

Topic 3.5.3: Dynamic Programming

1. Dynamic Programming is best defined as:

A) A method that repeatedly recalculates the same sub-problems

B) An optimization technique that breaks problems into simpler sub-problems and stores their results to avoid redundant calculations

C) A brute-force method of generating and testing all solutions

D) A method used only for sorting algorithms

2. Who coined the term "Dynamic Programming," and in what context?

A) Alan Turing, to describe the Halting Problem

B) Richard Bellman, while working at RAND Corporation in the 1950s

C) Stephen Cook, while formulating the P vs. NP question

D) Leonhard Euler, while solving the Seven Bridges of Königsberg

3. Dynamic Programming is particularly useful for problems with:

A) No relationship between sub-problems

B) Overlapping sub-problems and optimal substructure

C) Only a single possible sub-problem

D) Sub-problems that must never be reused

4. In the Fibonacci sequence example, what technique does Dynamic Programming use to avoid redundant calculations?

A) Backtracking through previous guesses

- B) Storing (memoizing) each calculated value for later reuse**
 - C) Discarding all previous results after each calculation
 - D) Generating random guesses until one is correct
5. Using the memoization table method described in this unit, what is the value of fib(4), given fib(0)=0, fib(1)=1, fib(2)=1, and fib(3)=2?
- A) 2
 - B) 3**
 - C) 5
 - D) 8
-

Topic 3.5.4: Backtracking

1. Backtracking is best described as:
- A) Solving a problem entirely at once, with no reconsideration of choices
 - B) Building a solution step by step, and reversing course whenever a path fails to lead to a solution**
 - C) Making irreversible locally optimal choices at each step
 - D) Storing all previous sub-problem results in a table
2. Backtracking is most commonly used for problems such as:
- A) Simple linear searches only
 - B) Puzzles or problems that require considering all possible combinations, like mazes**
 - C) Sorting a small list of five numbers
 - D) Calculating constant-time operations
-

Topic 3.6: Commonly Used Algorithms

1. According to this unit, sorting, searching, and graph traversal algorithms are best described as:
- A) Obsolete techniques no longer used in modern software
 - B) Foundational algorithms that serve as building blocks for more complex operations**
 - C) Techniques used only in artificial intelligence research
 - D) Algorithms that cannot be analyzed using Big O notation
-

Topic 3.6.1: Sorting Algorithms

1. Sorting algorithms are primarily used to:
 - A) Search for a specific item within a dataset
 - B) Arrange data in a particular order, such as ascending or descending**
 - C) Traverse a graph level by level
 - D) Verify whether a proposed solution is correct
 2. Sorting is often described as a prerequisite for which other type of algorithm discussed in this unit?
 - A) Greedy algorithms
 - B) Binary Search**
 - C) Backtracking
 - D) Dynamic Programming
-

Topic 3.6.1.1: Bubble Sort

1. Bubble Sort works by:
 - A) Selecting the minimum element and placing it at the front of the list
 - B) Repeatedly stepping through the list, comparing adjacent elements, and swapping them if in the wrong order**
 - C) Dividing the list into smaller sub-lists and merging them
 - D) Storing previously computed results in a memoization table
2. What is the time complexity of Bubble Sort?
 - A) $O(1)$
 - B) $O(\log n)$
 - C) $O(n)$
 - D) $O(n^2)$**
3. For the list [5, 3, 8, 4, 2], what are the first two elements compared in Bubble Sort's first pass?
 - A) 5 and 3**
 - B) 3 and 8
 - C) 8 and 4
 - D) 4 and 2
4. After the first full pass of Bubble Sort on an unsorted list, what is guaranteed?
 - A) The entire list is fully sorted
 - B) The largest element has moved to its correct final position at the end of the list**
 - C) The smallest element has moved to the front of the list
 - D) No further passes will ever be needed

5. Why is Bubble Sort considered useful despite its inefficiency for large datasets?
- A) It is the fastest known sorting algorithm
 - B) It is easy to understand and implement, making it useful for educational purposes and small datasets**
 - C) It always outperforms Merge Sort
 - D) It requires no comparisons at all
-

Topic 3.6.1.2: Selection Sort

1. Selection Sort works by:
 - A) Comparing adjacent elements and swapping them repeatedly
 - B) Selecting the smallest (or largest) element from the unsorted part of the list and swapping it with the first unsorted element**
 - C) Dividing the list into halves recursively
 - D) Applying a greedy coin-change strategy
 2. What is the time complexity of Selection Sort?
 - A) $O(1)$
 - B) $O(\log n)$
 - C) $O(n)$
 - D) $O(n^2)$**
 3. For the list [29, 10, 14, 37, 13], which element does Selection Sort identify as the minimum in the first round?
 - A) 29
 - B) 10**
 - C) 14
 - D) 13
 4. What happens to the boundary between sorted and unsorted sections after each round of Selection Sort?
 - A) It resets to the beginning of the list
 - B) It moves forward by one element**
 - C) It moves backward by one element
 - D) It disappears entirely
-

Topic 3.6.2: Search Algorithms

1. Search algorithms are designed to:
 - A) Arrange data in ascending or descending order
 - B) Find specific elements or a set of elements within a dataset**
 - C) Break a large problem into smaller sub-problems
 - D) Store memoized results of previous calculations
-

Topic 3.6.2.1: Linear Search

1. Linear Search works by:
 - A) Repeatedly dividing the list in half
 - B) Checking each item in the list one by one until the target is found**
 - C) Selecting the minimum element from the unsorted list
 - D) Comparing adjacent elements and swapping them
 2. What is the time complexity of Linear Search?
 - A) $O(1)$
 - B) $O(\log n)$
 - C) $O(n)$**
 - D) $O(n^2)$
 3. In the example list [Karachi, Lahore, Islamabad, Faisalabad], how many comparisons does Linear Search require to find "Islamabad"?
 - A) 1
 - B) 2
 - C) 3**
 - D) 4
 4. Does Linear Search require the list to be sorted beforehand?
 - A) Yes, always
 - B) No, it works on both sorted and unsorted lists**
 - C) Only if the list has more than 10 elements
 - D) Only for numeric data
-

Topic 3.6.2.2: Binary Search

1. Binary Search works by:
 - A) Checking each item one by one from the start of the list
 - B) Repeatedly dividing the search interval in half and discarding the half where the item**

cannot be

- C) Selecting the smallest unsorted element repeatedly
- D) Comparing every possible pair of elements

2. What is the time complexity of Binary Search?

- A) $O(1)$
- B) $O(\log n)$**
- C) $O(n)$
- D) $O(n^2)$

3. What is a required precondition for Binary Search to work correctly?

- A) The list must contain only even numbers
- B) The list must already be sorted**
- C) The list must have fewer than 10 elements
- D) The list must contain no duplicate values

4. In the example list [21, 34, 43, 57, 66, 78], what is the first middle element Binary Search checks while searching for 78?

- A) 21
- B) 43**
- C) 66
- D) 78

5. If the target value is larger than the current middle element in Binary Search, what happens next?

- A) The search ends immediately with no result
- B) The search continues on the right half of the list**
- C) The search continues on the left half of the list
- D) The entire list is searched again from the beginning

6. Which historical example is used in this unit to illustrate the real-world intuition behind Binary Search?

- A) Bank tellers processing transactions
 - B) Telephone directory operators finding names among millions of sorted entries**
 - C) Chefs following a recipe
 - D) Librarians cataloging books alphabetically
-

Topic 3.6.3: Graph Algorithms

1. Graph algorithms are used to explore and analyze:
 - A) Sorted arrays of numbers only
 - B) Graphs, which are data structures made up of nodes (vertices) connected by edges**
 - C) Recipes and cooking instructions
 - D) Complexity classes such as P and NP
 2. Who is credited with inventing graph theory by solving the Seven Bridges of Königsberg problem in 1736?
 - A) Alan Turing
 - B) Leonhard Euler**
 - C) Richard Bellman
 - D) Stephen Cook
-

Topic 3.6.3.1: Breadth-First Search (BFS)

1. Breadth-First Search (BFS) explores a graph:
 - A) As far down one branch as possible before backtracking
 - B) Level by level, starting from a given root node**
 - C) By randomly selecting nodes to visit
 - D) By comparing every pair of nodes
2. What data structure does BFS use to keep track of nodes that need to be explored?
 - A) A stack
 - B) A queue**
 - C) A memoization table
 - D) A Venn diagram
3. What is the time complexity of BFS?
 - A) $O(n \log n)$
 - B) $O(V)$
 - C) $O(V + E)$**
 - D) $O(n^2)$
4. BFS is particularly useful for finding:
 - A) The most memory-efficient path in a deep graph
 - B) The shortest path between two nodes in an unweighted graph**
 - C) The total number of edges in a graph only
 - D) Whether a number is prime

5. In the BFS process, what happens immediately after a node is dequeued and processed?
- A) The node is deleted from the graph permanently
 - B) All of its unvisited neighbors are enqueued**
 - C) The entire queue is emptied
 - D) The algorithm terminates
-

Topic 3.6.3.2: Depth-First Search (DFS)

1. Depth-First Search (DFS) explores a graph by:
 - A) Exploring all nodes at the same level before moving deeper
 - B) Exploring as far down a branch as possible before backtracking to explore other branches**
 - C) Selecting the smallest node value first
 - D) Comparing every possible pair of nodes
2. What data structure does DFS use to manage nodes that need to be explored?
 - A) A queue
 - B) A stack**
 - C) A memoization table
 - D) A linked list only
3. What is the time complexity of DFS?
 - A) $O(n \log n)$
 - B) $O(V)$
 - C) $O(V + E)$**
 - D) $O(n^2)$
4. According to this unit, which traversal is generally more memory-efficient for deep graphs?
 - A) BFS
 - B) DFS**
 - C) Both are equally memory-efficient in all cases
 - D) Neither is memory-efficient for deep graphs
5. DFS is commonly used for solving which type of problem, according to this unit?
 - A) Sorting a list of numbers
 - B) Puzzles like mazes, exploring one path until a dead end, then backtracking**
 - C) Verifying NP-Complete problems
 - D) Calculating Big O notation
6. What is the key defining behavior that distinguishes DFS from BFS?
 - A) DFS always finds the shortest path, while BFS does not
 - B) DFS commits to a single path as deep as possible before backtracking, while BFS**

expands outward level by level

- C) DFS uses a queue, while BFS uses a stack
 - D) DFS has a different time complexity than BFS
-

Section B: Short Answer Questions (SQs)

Topic 3.1: Understanding Computational Problems

1. Define a computational problem in your own words.
2. Identify and briefly describe the three essential parts of every computational problem.
3. Explain briefly why "process" is a necessary component of a computational problem, not just input and output.

Topic 3.1.1: Characterizing Computational Problems

1. Explain briefly what it means to "characterize" a computational problem.
2. List the three things that must be identified before a problem can be solved computationally.

Topic 3.1.1.1: Classifying Computational Problems

1. Define a Decision Problem and give a brief example.
2. Differentiate between a Search Problem and an Optimization Problem.
3. Define a Counting Problem and give a brief example.
4. Identify which classification ("Decision," "Search," "Optimization," or "Counting") best fits the question: "What is the fastest route from Karachi to Multan?"

Topic 3.1.1.2: Well-defined vs. Ill-defined Problems

1. Differentiate between a well-defined problem and an ill-defined problem.
2. List the four characteristics that make a problem well-defined.
3. Explain briefly why "How to reduce poverty in Pakistan" is considered an ill-defined problem.
4. Predict whether "Sort a list of 10 integers in ascending order" is well-defined or ill-defined, and briefly justify your answer.

Topic 3.2: Algorithms for Problem Solving

1. Define an algorithm in your own words.
2. Explain briefly why algorithms are described as being like a "recipe."
3. State one real-world application of algorithms mentioned in this unit, such as PageRank.

Topic 3.2.1: Generate-and-Test Method

1. Define the Generate-and-Test method.
2. List the scenarios in which the Generate-and-Test method is particularly useful.
3. Explain briefly the roles of the "Generator" and the "Tester" in this method.
4. Predict the outcome if the Generate-and-Test method is applied to a problem with an extremely large problem space and no heuristics.

Topic 3.3: Problem Solvability and Complexity

1. Explain briefly the difference between asking "can this problem be solved?" and "how efficiently can this problem be solved?"

Topic 3.3.1: Solvable vs. Unsolvable Problems

1. Differentiate between a solvable problem and an unsolvable problem.
2. Define the Halting Problem in your own words.
3. Explain briefly why Alan Turing's proof about the Halting Problem is considered significant in computer science.
4. Identify one example of a solvable problem discussed in this unit, and briefly explain why it is solvable.

Topic 3.3.2: Tractable vs. Intractable Problems

1. Differentiate between a tractable problem and an intractable problem.
2. Explain briefly why the Traveling Salesman Problem (TSP) is considered intractable.
3. State the time complexity associated with tractable problems.
4. Predict what might happen to the runtime of an intractable algorithm as its input size grows from 10 to 100.

Topic 3.3.3: Complexity Classes (P, NP, NP-hard, NP-complete)

1. Explain briefly the significance of the P vs. NP question in computer science.
2. Identify the two computer scientists credited with independently formulating the P vs. NP question.
3. State the prize amount offered for solving the P vs. NP problem, and by which organization.

Topic 3.3.3.1: Class P

1. Define Class P.
2. Give a brief example of a problem that belongs to Class P.

Topic 3.3.3.2: Class NP

1. Define Class NP.
2. Explain briefly why solving a Sudoku puzzle is used as an example of Class NP.

Topic 3.3.3.3: Class NP-Hard

1. Define NP-hard problems.
2. Identify one example of an NP-hard problem discussed in this unit.

Topic 3.3.3.4: NP-Complete

1. Define NP-Complete problems.
2. Differentiate between NP-hard and NP-Complete problems.
3. Explain briefly the significance of the Knapsack Problem as an example of an NP-Complete problem.

Topic 3.4: Algorithm Analysis

1. Define algorithm analysis.
2. Explain briefly why algorithm analysis is important when selecting an algorithm for a task.

Topic 3.4.1: Time Complexity

1. Define time complexity in your own words.
2. Explain briefly how time complexity helps predict an algorithm's performance on larger datasets.

Topic 3.4.1.1: Big O Notation

1. Define Big O notation.
2. Differentiate between $O(1)$ and $O(n)$ time complexity, with a brief example of each.
3. Differentiate between $O(n)$ and $O(n^2)$ time complexity, with a brief example of each.
4. Explain briefly why $O(\log n)$ time complexity grows more slowly than $O(n)$.
5. Predict the output/classification: an algorithm containing a single loop that runs once through a list of n items — what is its Big O time complexity?
6. Predict the output/classification: an algorithm containing two nested loops, each running through a list of n items — what is its Big O time complexity?

Topic 3.4.2: Space Complexity

1. Define space complexity in your own words.
2. Differentiate between time complexity and space complexity.

Topic 3.5: Algorithm Design Techniques

1. Explain briefly why algorithm design techniques are described as "reusable strategies."
2. List the four algorithm design techniques covered in this unit.

Topic 3.5.1: Divide and Conquer

1. Define the Divide and Conquer technique.
2. Explain briefly how Merge Sort demonstrates the Divide and Conquer approach.
3. Identify the type of problems for which Divide and Conquer is particularly effective.

Topic 3.5.2: Greedy Algorithms

1. Define a Greedy algorithm.
2. Explain briefly what "optimal substructure" means in the context of Greedy algorithms.
3. State one limitation of Greedy algorithms mentioned in this unit.
4. Map out, in words, how a Greedy algorithm would solve the Coin Change problem for a given amount.

Topic 3.5.3: Dynamic Programming

1. Define Dynamic Programming.
2. Explain briefly what "overlapping sub-problems" means.
3. Identify the historical figure credited with coining the term "Dynamic Programming," and briefly state the context in which it was coined.
4. Explain briefly how memoization helps avoid redundant calculations in the Fibonacci sequence example.

Topic 3.5.4: Backtracking

1. Define Backtracking in your own words.
2. Explain briefly how Backtracking differs from the Generate-and-Test method.
3. Identify one type of problem for which Backtracking is commonly used.

Topic 3.6: Commonly Used Algorithms

1. Explain briefly why sorting, searching, and graph algorithms are described as "foundational" algorithms.

Topic 3.6.1: Sorting Algorithms

1. Define a sorting algorithm.

2. Explain briefly why sorting is often considered a prerequisite for other operations, such as searching.

Topic 3.6.1.1: Bubble Sort

1. Define Bubble Sort and briefly describe its process.
2. State the time complexity of Bubble Sort.
3. Predict the result after one full pass of Bubble Sort on the list [9, 3, 6, 1].
4. Explain briefly why Bubble Sort is considered inefficient for large datasets, despite being easy to understand.

Topic 3.6.1.2: Selection Sort

1. Define Selection Sort and briefly describe its process.
2. State the time complexity of Selection Sort.
3. Differentiate between Bubble Sort and Selection Sort in terms of how they identify which elements to move.

Topic 3.6.2: Search Algorithms

1. Define a search algorithm.
2. State one real-world application of search algorithms mentioned in this unit.

Topic 3.6.2.1: Linear Search

1. Define Linear Search and briefly describe its process.
2. State the time complexity of Linear Search.
3. Predict the number of comparisons Linear Search would need, in the worst case, to search a list of 50 items.

Topic 3.6.2.2: Binary Search

1. Define Binary Search and briefly describe its process.
2. State the time complexity of Binary Search.
3. Identify the required precondition for Binary Search to function correctly.
4. Predict the middle element Binary Search would check first in the sorted list [2, 4, 6, 8, 10, 12, 14].

Topic 3.6.3: Graph Algorithms

1. Define a graph in the context of graph algorithms.
2. Identify the historical figure and puzzle credited with the origin of graph theory.

Topic 3.6.3.1: Breadth-First Search (BFS)

1. Define BFS and briefly describe its process.
2. State the data structure used by BFS and the time complexity of BFS.
3. Explain briefly why BFS is well-suited for finding the shortest path in an unweighted graph.

Topic 3.6.3.2: Depth-First Search (DFS)

1. Define DFS and briefly describe its process.
 2. State the data structure used by DFS and the time complexity of DFS.
 3. Differentiate between BFS and DFS in terms of the data structure each uses and the traversal pattern each follows.
-

Section C: Long / Extensive Questions (LQs)

Topic 3.1: Understanding Computational Problems

1. Analyze the concept of a computational problem in detail.
 - a) Define a computational problem and explain the role of input, process, and output.
 - b) Construct an original example of a computational problem (not used in the textbook) and clearly identify its input, process, and output.
 - c) Discuss why precisely identifying these three components is essential before attempting to design an algorithm.

Topic 3.1.1 & 3.1.1.1: Characterizing and Classifying Computational Problems

1. Elaborate on the process of characterizing and classifying computational problems.
 - a) Explain, in detail, the steps involved in characterizing a computational problem.
 - b) Discuss the four classifications of computational problems (Decision, Search, Optimization, Counting), providing one original real-world example of each that is different from the textbook's examples.
 - c) Evaluate why correctly classifying a problem before attempting to solve it can influence the choice of algorithm design technique used later.

Topic 3.1.1.2: Well-defined vs. Ill-defined Problems

1. Discuss in detail the distinction between well-defined and ill-defined computational problems.
 - a) Define both categories, listing all defining characteristics of each.
 - b) Construct two original examples: one well-defined problem and one ill-defined problem, and

justify your classification of each using the four-part checklist (goal, input, process, output).

c) Analyze how an ill-defined problem might be broken down or reframed into a well-defined problem suitable for computational solving.

Topic 3.2 & 3.2.1: Algorithms and the Generate-and-Test Method

1. Discuss in detail the role of algorithms in computational problem-solving, focusing on the Generate-and-Test method.
 - a) Explain what an algorithm is and why it is fundamental to software development.
 - b) Construct a step-by-step trace of the Generate-and-Test method being used to solve a simple original problem of your choice (e.g., guessing a hidden number, or finding a combination).
 - c) Evaluate the strengths and weaknesses of the Generate-and-Test method, and discuss the conditions under which it becomes impractical.
-

Topic 3.3: Problem Solvability and Complexity

1. Analyze, in detail, the concept of problem solvability, using the Halting Problem as a central case study.
 - a) Explain what makes a problem "solvable" versus "unsolvable," using clear definitions.
 - b) Discuss in detail why the Halting Problem is considered unsolvable, including an explanation of what Alan Turing proved in 1936.
 - c) Evaluate the broader implications of unsolvable problems for the field of computer science — are there practical consequences of knowing a problem is unsolvable?
2. Discuss in detail the distinction between tractable and intractable problems.
 - a) Define tractable and intractable problems, including their associated time complexities.
 - b) Using the Traveling Salesman Problem (TSP) as a case study, explain why it is classified as intractable, and describe how the number of possible solutions grows as the number of cities increases.
 - c) Evaluate the real-world implications of intractability — how might businesses or researchers work around a problem that is technically unsolvable in reasonable time for large inputs?

Topic 3.3.3: Complexity Classes (P, NP, NP-hard, NP-complete)

1. Construct a comprehensive comparative analysis of the four complexity classes discussed in this unit: P, NP, NP-hard, and NP-Complete.
 - a) Define each of the four classes clearly, and explain the relationship between them (referencing the Venn diagram structure).
 - b) Provide one original example problem for each class, different from the textbook's examples, and justify why each example belongs in its respective class.
 - c) Discuss the historical origin and significance of the P vs. NP question, including the

Millennium Prize, and evaluate why this question remains unresolved.

d) Evaluate what it would mean for global cybersecurity and cryptography if P were proven equal to NP.

Topic 3.4: Algorithm Analysis (Time and Space Complexity)

1. Discuss in detail the process of algorithm analysis, focusing on time and space complexity.
 - a) Define time complexity and space complexity, and explain how each is used to evaluate an algorithm.
 - b) Construct a detailed explanation of Big O notation, covering $O(1)$, $O(\log n)$, $O(n)$, and $O(n^2)$, with an original example of an algorithm for each category.
 - c) Analyze a scenario in which two algorithms solve the same problem, but one has $O(n)$ time complexity and the other has $O(n^2)$ time complexity. Discuss how each algorithm's performance would differ as the input size grows from 100 to 100,000 items.
 - d) Evaluate the trade-offs a developer might face when an algorithm has excellent time complexity but poor space complexity, or vice versa.
-

Topic 3.5: Algorithm Design Techniques

1. Construct a comprehensive comparative analysis of the four algorithm design techniques covered in this unit: Divide and Conquer, Greedy Algorithms, Dynamic Programming, and Backtracking.
 - a) Define each technique and explain its core strategy in detail.
 - b) For each technique, provide one original example problem (different from the textbook) that the technique could be used to solve, and justify why that technique is well-suited to the problem.
 - c) Discuss, using the Coin Change problem, why a Greedy algorithm might fail to find the optimal solution for certain coin denomination systems, even though it works for others.
 - d) Evaluate the relationship between Dynamic Programming and Greedy algorithms — under what conditions might a problem be solvable by both, and under what conditions would only Dynamic Programming guarantee the correct answer?
 2. Design and trace a Dynamic Programming solution in detail.
 - a) Explain the concept of memoization and why it is central to Dynamic Programming.
 - b) Construct a full memoization trace table for calculating $\text{fib}(8)$, showing every intermediate value calculated and stored, following the same format used in this unit's Fibonacci walkthrough.
 - c) Discuss how many redundant calculations a plain (non-memoized) recursive approach would perform compared to the memoized approach, and evaluate the overall efficiency gain.
-

Topic 3.6.1: Sorting Algorithms

1. Discuss in detail the mechanics, efficiency, and trade-offs of Bubble Sort and Selection Sort.
 - a) Explain the step-by-step process of both Bubble Sort and Selection Sort.
 - b) Construct a full, pass-by-pass trace table for sorting the list [8, 4, 6, 2, 9] using Bubble Sort, showing every comparison, swap decision, and the resulting list state after each pass.
 - c) Construct a full, round-by-round trace table for sorting the same list [8, 4, 6, 2, 9] using Selection Sort, showing the minimum element identified and the resulting list state after each round.
 - d) Evaluate and compare the total number of comparisons and swaps required by each algorithm for this list, and discuss which algorithm might be preferable in a scenario where swapping is a computationally expensive operation.

Topic 3.6.2: Search Algorithms

1. Discuss in detail the differences between Linear Search and Binary Search.
 - a) Explain the process and preconditions of both Linear Search and Binary Search.
 - b) Construct a full step-by-step trace table showing Linear Search locating the value 55 in the unsorted list [12, 87, 55, 3, 41].
 - c) Construct a full step-by-step trace table showing Binary Search locating the value 55 in the sorted list [3, 12, 41, 55, 87], including the middle element chosen and the half of the list eliminated at each step.
 - d) Evaluate how the performance gap between Linear Search and Binary Search would change as the dataset grows from 10 items to 10,000,000 items, referencing their respective time complexities.

Topic 3.6.3: Graph Algorithms

1. Construct a comprehensive comparative analysis of Breadth-First Search (BFS) and Depth-First Search (DFS).
 - a) Explain the process of both BFS and DFS, including the data structure (queue or stack) each relies on.
 - b) Construct an original graph of at least 6 nodes (describe its structure in words or as an edge list), and produce a full step-by-step trace table for both a BFS and a DFS traversal starting from the same root node, showing the queue/stack state and the order in which nodes are processed.
 - c) Discuss the time complexity of both algorithms, and evaluate the scenarios in which BFS would be preferable to DFS, and vice versa (e.g., shortest-path finding vs. maze-solving vs. memory-constrained environments).
 - d) Evaluate the historical significance of Leonhard Euler's Seven Bridges of Königsberg

problem in establishing the foundation of graph theory, and discuss how that 18th-century puzzle connects conceptually to modern applications like GPS navigation or social network analysis.
